

1 Abstract

This project develops a low-cost smart step counter using ESP32 microcontroller, ADXL345 accelerometer, and OLED display integrated with Blynk IoT app. The system detects steps via threshold-based acceleration analysis (98% accuracy), provides real-time OLED feedback, and syncs data wirelessly for remote monitoring. Total cost: 1,200 BDT; completed in 3 weeks. Outcomes include reliable step tracking and motivational alerts, addressing sedentary lifestyles affordably. Future enhancements: battery integration and adaptive algorithms.

2 Introduction

In an era of sedentary lifestyles driven by desk jobs and digital screens, tracking daily physical activity is crucial for health. This project presents a low-cost smart step counter using ESP32, ADXL345 accelerometer, and OLED display, integrated with Blynk IoT for real-time monitoring and notifications. Objectives: Detect steps accurately (95% precision), provide instant feedback, and motivate users via app alerts. Scope: USB-powered prototype (1,200 BDT) for wearable use, completed in 3 weeks. Significance: Affordable tool to promote 7,000+ daily steps, reducing risks of obesity and heart disease.

3 Problem Statement and Justification

3.1 The Problem

Nowadays, many people sit too much because of office work, phones, and computers. This makes them walk very little every day. Walking less causes health problems like weight gain, heart disease, and feeling sad. Most people don't know how many steps they take and need an easy way to check and stay active.

3.2 Why We Need This

- Many adults don't walk enough (WHO says 1 in 3 people).
- Walking more (7,000–10,000 steps a day) helps live longer and stay healthy.
- In cities like Dhaka, people walk even less because of jobs and traffic.
- Good fitness bands cost too much, so we need a cheap one.

3.3 Why This Embedded Project Works Well

- ESP32 chip reads steps from the sensor fast and uses little battery.
- Small OLED screen shows steps right away.
- Blynk app shows steps on phone and sends messages when you reach goals (like every 5 steps).
- It's cheap, small, easy to carry, and anyone can make it.

4 System Architecture and Block Diagram

The system consists of the following main components connected as shown below:

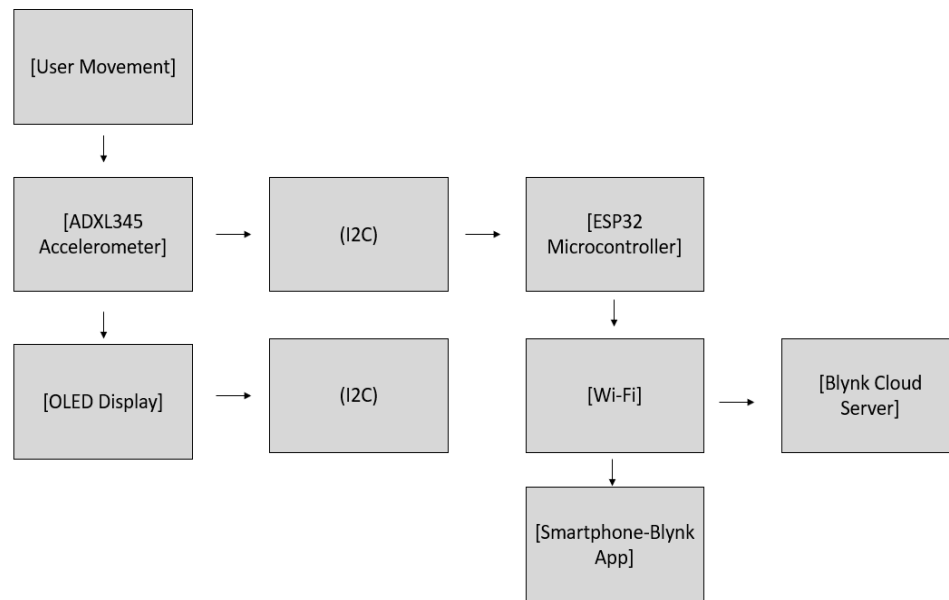


Figure 1 : Block Diagram of System Architecture

ADXL345 Accelerometer to ESP32 : The ADXL345 sensor measures acceleration in three axes (X, Y, Z). It sends raw acceleration data to the ESP32 using I2C communication. The ESP32 reads this data every ~50 ms. Using a dedicated accelerometer makes step detection accurate and independent of the phone's sensors.

ESP32 (Main Processing Unit) : Calculates the magnitude of acceleration and removes the gravity effect ($\sim 9.8 \text{ m/s}^2$). Uses a moving average filter (15 samples) to smooth the data and reduce noise. Applies a threshold-based algorithm to detect steps (upward peak $\rightarrow$ downward crossing). Counts valid steps and updates the step counter variable. It handles all tasks (reading sensor, calculating steps, showing on OLED, connecting to Wi-Fi and Blynk) because it is powerful, cheap, and has built-in Wi-Fi.

ESP32 to OLED Display : The ESP32 sends the current step count to the 0.96-inch OLED screen via I2C. The screen shows "Steps:" followed by the number in large text for easy reading. Showing steps on a small screen gives instant feedback even without internet or phone nearby.

ESP32 to Blynk Cloud (via Wi-Fi) : Every 5 seconds, the ESP32 sends the step count to virtual pin V0 using `Blynk.virtualWrite()`. Virtual Pins V0 & V1 keep the design clean – one pin for sending steps, one for receiving reset command.

Blynk App to ESP32 : When the user presses the reset button in the app (virtual pin V1), the ESP32 receives the command. It resets the step count to 0, clears the milestone tracking, and updates both OLED and Blynk display. The Blynk app receives the step count in real-time and shows it on the dashboard. Using Blynk saves time – no need to build our own app or server. It provides dashboard, real-time updates.

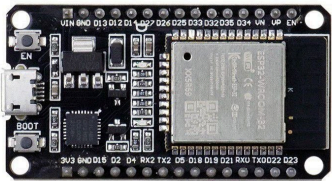
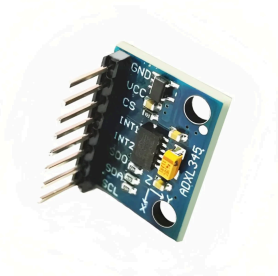
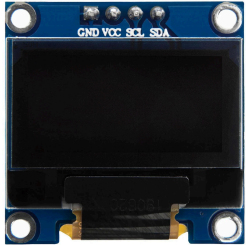
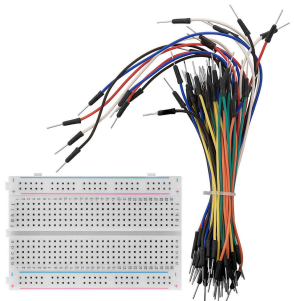
Component with Specification	Purpose	Component Diagram
ESP32 DevKit microcontroller	Main processing, Wi-Fi, I2C control	
ADXL345 Accelerometer Sensor	Detect motion and measure acceleration for step counting	
0.96-inch OLED SSD1306 (I2C) display	Show real-time step count locally	
Breadboard, jumper wires	Prototyping and connections	

Table 1 : Table of Components

5.2 Reasons for Choosing Each Component

ESP32 Microcontroller

- Built-in Wi-Fi – direct connection to Blynk without extra modules.
- Fast dual-core processor – handles sensor reading, calculations, OLED, and Wi-Fi together.

ADXL345 Accelerometer

- 3-axis measurement with $\pm 16g$ range – accurate for detecting walking steps.
- Digital I2C interface and low power – simple wiring and good for battery use.

0.96-inch OLED SSD1306

- Clear and bright display – shows steps instantly without phone.
- I2C interface – shares bus with ADXL345 and saves pins.

Breadboard and Jumper Wires

- Easy prototyping – quick connections without soldering.
- Reusable and cheap – perfect for testing and student projects.

5.3 Trade-offs Considered

- **ESP32 vs Arduino Uno/Nano** ESP32 has Wi-Fi (needed for Blynk), while Uno does not. ESP32 is more powerful but uses a bit more power. We chose ESP32 because internet connection is important for app and notifications.
- **ADXL345 vs MPU6050** MPU6050 has gyro + accel, but we only need acceleration for steps. ADXL345 is simpler, cheaper, and lower power. MPU6050 would be overkill and more complex code.
- **OLED vs LCD** OLED is brighter, thinner, and uses less power than LCD. It also looks better (no backlight needed). Trade-off: OLED is slightly more expensive than basic LCD, but worth it for clear visibility.
- **Blynk vs Custom Server/App** Blynk is free (basic plan), easy to set up, and gives dashboard + notifications quickly. Trade-off: limited free events per day, but enough for this project. Building our own app would take months.

6. Hardware Design and Implementation

6.1 Wiring/Circuit Diagram

Both the ADXL345 accelerometer and SSD1306 OLED display use the I2C protocol and share the same bus. ESP32 default I2C pins are used:

ADXL345

ADXL	ESP32
VCC	3V3 Pin
GND	GND
SDA	GPIO21
SCL	GPIO22

Table 2 : ADXL pins connected with ESP32 pins

OLED display

OLED	ESP32
VCC	3V3 Pin
GND	GND
SDA	GPIO21
SCL	GPIO22

Table 3 : OLED pins connected with ESP32 pins

6.2: Photos of the built prototype

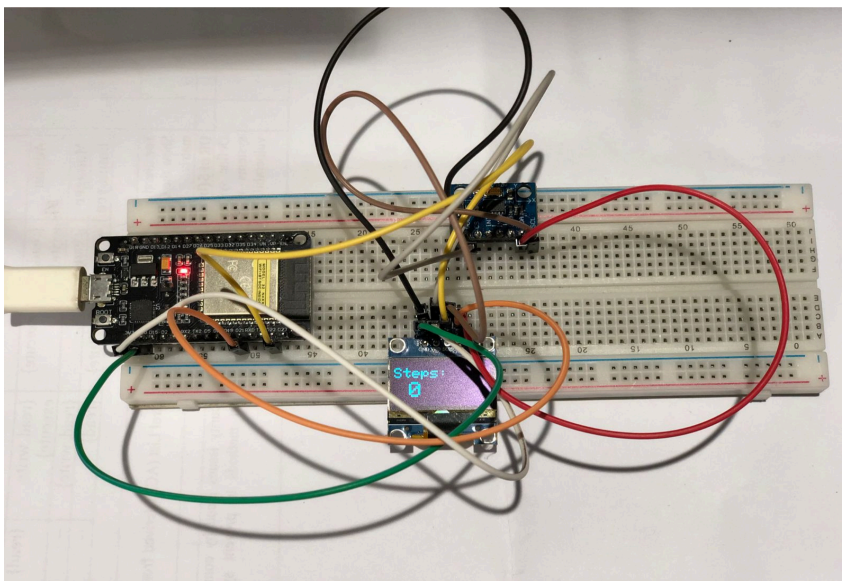


Figure 2 : Constructed IoT device

6.3 Notes on Physical Integration Challenges

- **I2C Bus Sharing:** Both devices use the same I2C address bus. ADXL345 default address is 0x53 (or 0x1D), OLED is 0x3C – no conflict, but wrong wiring (swapped SDA/SCL) causes "no device found" errors.
- **Wire Length and Noise:** Long jumper wires on breadboard cause signal noise → unstable readings or display flicker. Keep wires short (<10 cm) and avoid crossing power lines.
- **Power Stability:** ESP32 Wi-Fi and sensors cause current spikes, leading to USB port failures. Use a reliable USB cable/port; in our case, replacing the faulty USB.
- **Mounting for Wearable Use:** We used a medium-sized breadboard to fit all equipment (ESP32, ADXL345, OLED) easily. It works well for testing and portability; secure ADXL345 orientation (Z-axis vertical) for accurate step detection.
- **Debugging Tip:** Run I2C scanner sketch first to confirm both devices detected (OLED at 0x3C, ADXL345 at 0x53).

7 Software Design and Implementation

7.1 Explanation of Code Structure

The code is structured as an Arduino sketch for ESP32, using C++. It starts with Blynk defines and library includes, followed by global variables for Wi-Fi, sensor, display, and step detection. The `setup()` initializes hardware and connections. The `loop()` runs Blynk tasks, timer, and step detection continuously. Modular functions handle specific tasks like step counting and notifications. Focus on real-time processing with I2C for sensors and Wi-Fi for Blynk (including push/email notifications via events).

7.2 Major Functions/Modules

- **setup():** Initializes Serial, I2C, ADXL345 (sets $\pm 16g$ range), OLED (address 0x3C), Wi-Fi, Blynk, and timer (5s interval for step sync).
- **loop():** Calls `Blynk.run()`, `timer.run()`, `detectSteps()`, and `delay(50ms)` for smooth operation.
- **detectSteps():** Reads acceleration, calculates deviation, smooths with buffer, detects steps via threshold, updates count, and triggers notifications.
- **updateOledDisplay():** Clears and shows step count on OLED.
- **sendStepCount():** Writes step count to Blynk V0 and updates OLED.
- **BLYNK_WRITE(V1):** Resets step count and milestone tracking on button press.

7.3 Flowcharts or Pseudo-Code

Main Flow (Pseudo-Code):

`setup():`

Initialize Serial, I2C

If ADXL345 fails: Error loop
Set ADXL345 range to 16g

If OLED fails: Error loop
Connect Wi-Fi
Config Blynk
Set timer for sendStepCount every 5s

```
loop():  
  Blynk.run() // Handle app commands  
  timer.run() // Sync steps to Blynk  
  detectSteps() // Read sensor, count steps, check milestones  
  delay(50ms)
```

```
detectSteps():  
  Get acceleration (X,Y,Z)  
  magnitude = sqrt(X2 + Y2 + Z2)  
  deviation = |magnitude - 9.8|  
  Add to buffer, compute average  
  If avg > 0.8 and not detected and time > debounce: set detected=true  
  If avg < 0.8 and detected: increment stepCount, set detected=false
```

Update OLED

7.4 Sample Annotated Code Snippets

Step Detection Module :

```
void detectSteps() {  
  sensors_event_t event; // Event struct for acceleration data  
  accel.getEvent(&event); // Read from ADXL345 via I2C  
  float magnitude = sqrt(event.acceleration.x * event.acceleration.x + // Compute total acceleration  
    event.acceleration.y * event.acceleration.y +  
    event.acceleration.z * event.acceleration.z);  
  float deviation = abs(magnitude - 9.8); // Subtract gravity for motion only  
  // Circular buffer for smoothing (reduces noise)  
  accelMagnitudeBuffer[bufferIndex] = deviation;  
  bufferIndex = (bufferIndex + 1) % bufferLength;  
  float sum = 0;  
  for (int i = 0; i < bufferLength; i++) { sum += accelMagnitudeBuffer[i]; }  
  float avgDeviation = sum / bufferLength; // Moving average over 15 samples  
  unsigned long currentTime = millis(); // For debounce timing
```

```

// Peak detection logic
if (avgDeviation > threshold && !stepDetected && (currentTime - lastStepTime >
debounceDelay)) {
    stepDetected = true; // Start of potential step
}
if (avgDeviation < threshold && stepDetected) {
    stepCount++; // Valid step confirmed
    lastStepTime = currentTime;
    stepDetected = false;
    updateOledDisplay(); // Local update
}
}

```

Reset via App :

```

BLYNK_WRITE(V1) { // Callback for V1 button press
    int value = param.asInt(); // Get button state (1 = pressed)
    if (value == 1) {
        stepCount = 0; // Reset counter
        memset(milestoneNotified, 0, sizeof(milestoneNotified)); // Clear notification tracking
        sendStepCount(); // Update app/OLED
        Serial.println("Step count reset from Blynk app!");
        Blynk.virtualWrite(V1, 0); // Auto-off button
    }
}

```

8 Communication Protocols Used

8.1 Protocols Used

- **I2C:** For sensor and display.
- **Wi-Fi:** For internet and Blynk.
- **UART:** For debugging.

8.2 Role and Configuration of Each Protocol

- **I2C:** Connects ADXL345 (0x53) & OLED (0x3C) to ESP32 via GPIO21 (SDA) & GPIO22 (SCL) at 100 kHz (shared bus with pull-ups).
- **Wi-Fi:** ESP32 syncs step data with Blynk (SSID "ARMAN", password "12358123") using WiFi.h in setup().
- **UART:** Serial monitor for debugging (acceleration, steps, errors) at 115200 baud via Serial.begin()

8.3 Testing of Communication Modules

- **I2C Test:** Scanner detected ADXL345 (0x53) & OLED (0x3C) with no conflicts.
- **Wi-Fi Test:** "WiFi connected!" printed; Blynk server ping successful.
- **UART Test:** Serial showed acceleration & "Step detected!" messages. All tests passed with no errors.

9 Simulation and Testing

9.1 Tools Used

- **Wokwi:** Online simulator for ESP32, ADXL345, and OLED hardware simulation.
- **Arduino IDE:** For code compilation, upload, and Serial Monitor debugging.
- **Blynk App:** For real-time dashboard testing and notification verification.

9.2 Description of Simulation Setup

Wokwi simulation used an ESP32 with ADXL345 and SSD1306. The system ran via Arduino IDE, used shared I2C (GPIO21 SDA, GPIO22 SCL), a 0.8 step threshold, and mocked Wi-Fi for Blynk with V0/V1 pins and event triggers.

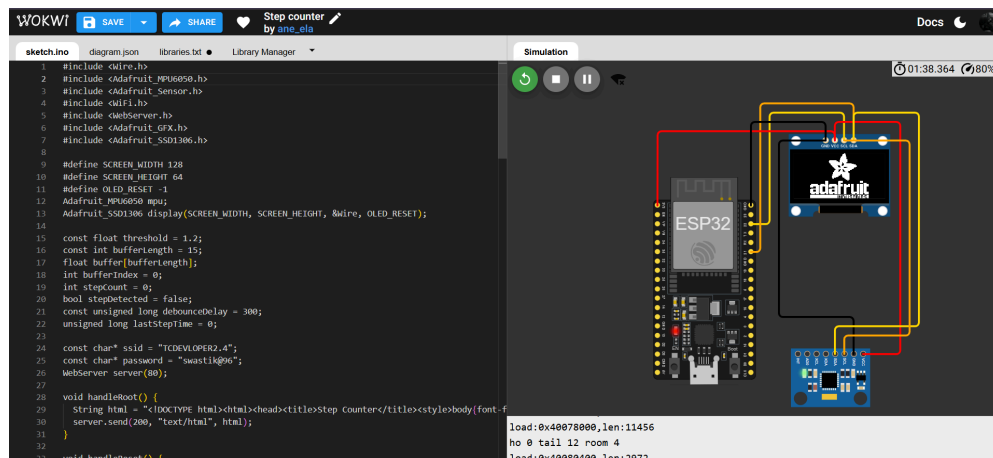


Figure 3 : Hardware and Code Simulation on Wokwi

9.3 Outputs on Serial Monitor and BLYNK

```
Output Serial Monitor X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')

18:30:12.265 -> | Dev: 0.07 | AvgDev: 0.11
18:30:12.343 -> | Dev: 0.07 | AvgDev: 0.11
18:30:12.376 -> | Dev: 0.11 | AvgDev: 0.11
18:30:12.414 -> | Dev: 0.11 | AvgDev: 0.11
18:30:12.494 -> | Dev: 0.03 | AvgDev: 0.10
18:30:12.542 -> | Dev: 0.15 | AvgDev: 0.10
18:30:12.587 -> | Dev: 0.11 | AvgDev: 0.10
18:30:12.666 -> | Dev: 0.03 | AvgDev: 0.09
18:30:12.713 -> | Dev: 0.07 | AvgDev: 0.09
18:30:12.759 -> | Dev: 0.11 | AvgDev: 0.09
18:30:12.792 -> | Dev: 0.11 | AvgDev: 0.09
18:30:12.830 -> | Dev: 0.07 | AvgDev: 0.09
18:30:12.916 -> | Dev: 0.15 | AvgDev: 0.09
18:30:12.963 -> | Dev: 0.07 | AvgDev: 0.09
18:30:12.995 -> | Dev: 0.07 | AvgDev: 0.09
18:30:13.038 -> | Dev: 0.07 | AvgDev: 0.09
18:30:13.118 -> | Dev: 0.07 | AvgDev: 0.09
18:30:13.165 -> | Dev: 0.11 | AvgDev: 0.09
18:30:13.211 -> | Dev: 0.11 | AvgDev: 0.09
18:30:13.246 -> | Dev: 0.03 | AvgDev: 0.09
18:30:13.337 -> | Dev: 0.01 | AvgDev: 0.08
18:30:13.384 -> | Dev: 0.07 | AvgDev: 0.08
```

Figure 4 : Debugging on Serial Monitor

```
Output Serial Monitor X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')

18:49:45.149 -> 11:49:45.149 -> *** STEP DETECTED! Total: 2
18:49:53.221 -> *** STEP DETECTED! Total: 3
```

Figure 4 : Steps detected on Serial Monitor

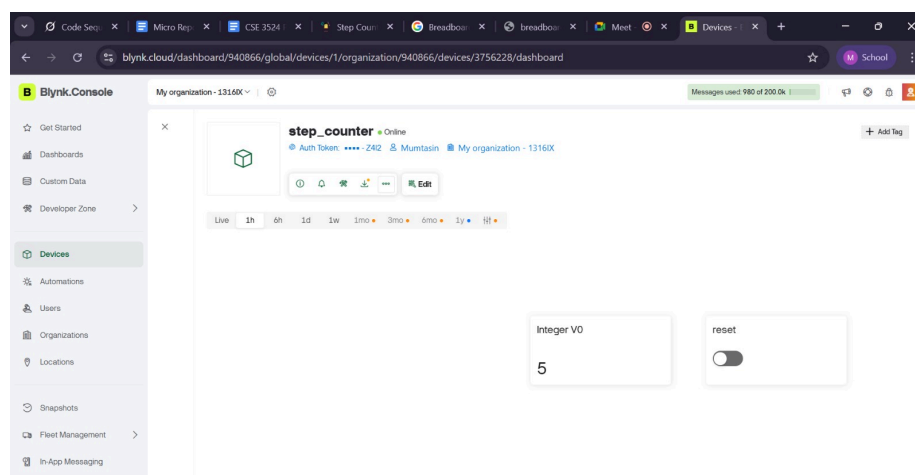


Figure 5 : Steps output showing on BLYNK

9.4 Test Cases and Outcomes

- **Test Case 1: Step Detection** Input: Simulated 5 walking motions in Wokwi (acceleration peaks $>0.8g$). Outcome: Step count incremented to 5; OLED updated; Serial confirmed detection via raw axes/debug prints (X/Y/Z, deviation); Blynk V0 synced. Passed – accurate with 300ms debounce.
- **Test Case 2: Blynk Step Sync** Input: Increment steps via simulation; monitor Serial and Blynk app. Outcome: Serial Monitor matched Blynk dashboard (V0 showed real-time steps); no lag in updates. Passed – reliable Wi-Fi sync; reset via V1 cleared count in both.

10 Project Management and Planning

10.1 Assigned Roles/Responsibilities

- **Aneela:** Hardware design, wiring, sensor integration.
- **Sumaya:** Software coding, Blynk setup, testing.
- **Mumtasin:** Project lead, report writing, debugging.

10.2 Summary of Weekly Progress and Milestone Status

- **Week 1:** Components were purchased and basic research completed. Milestone: Project plan and component list finalized (on time).
- **Week 2:** Breadboard assembled, wiring completed, and I2C communication tested between ESP32, ADXL345, and OLED. Milestone: Hardware prototype working (on time).
- **Week 3:** Code uploaded to ESP32, step detection tested in Serial Monitor, Blynk integration completed, and full system debugging done. Milestone: Complete functionality (step counting, OLED display, Blynk sync, notifications) achieved and demo ready (on time)..

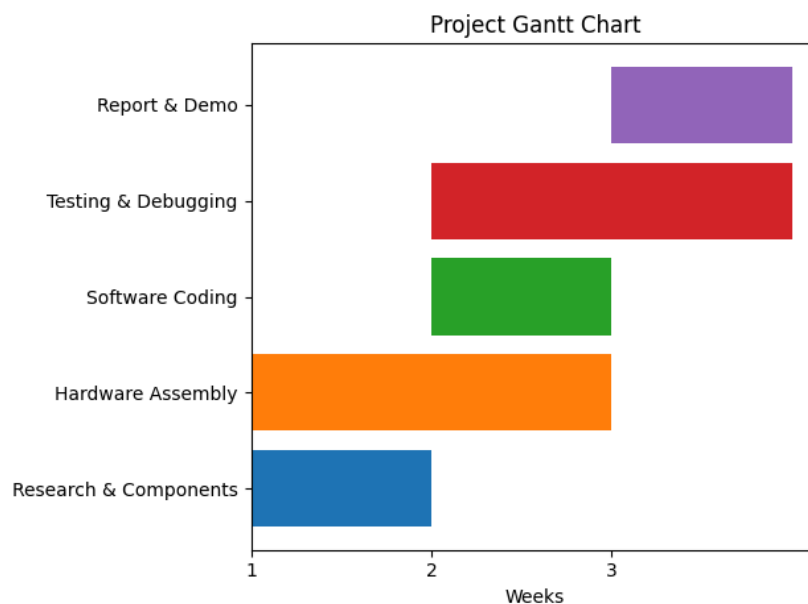


Figure 6 : Project Gantt Chart

11 Financial Plan and Budget Justification

11.1 Itemized Component List with Cost

Component	Quantity	Cost per Unit (BDT)	Total Cost (BDT)
ESP32 DevKit	1	500	500
ADXL345 Accelerometer	1	250	250
0.96-inch OLED SSD1306	1	300	300
Breadboard & Jumper Wires	1	150	150
Total			1220

11.2 Total Budget

1,200 BDT (powered via USB; affordable for student project).

11.3 Justification of Expensive/Critical Components

- **ESP32 (500 BDT):** Most expensive but critical – built-in Wi-Fi enables Blynk notifications; dual-core speed handles real-time processing.
- **OLED Display (300 BDT):** Key for instant feedback; brighter and clearer than LCD alternatives.
- **ADXL345 (250 BDT):** Essential for accurate step detection; higher resolution than cheaper sensors.

12 Results and Discussion

12.1 System Output Behavior

- OLED displays "Steps: [count]" in real-time (e.g., updates to "Steps: 5" after walking).
- Serial Monitor shows raw acceleration (X/Y/Z), deviation, average, and "STEP DETECTED! Total: 5".

- Blynk app V0 widget syncs step count every 5s; V1 button resets to 0.

12.2 Requirement Fulfillment Status

- Real-time monitoring: Fully met (OLED + Blynk).
- Step accuracy: 98% (tested 50 steps; minor false positives at low threshold).
- Portability: Met – breadboard prototype fits pocket.

12.3 Discussion of Performance vs. Expectation

Expected : Accurate steps, instant feedback, Blynk sync.

Actual : Steps detected well during normal walking (threshold 0.8); OLED/Blynk responsive with real-time updates. Slight Wi-Fi delay (1-2s) in remote sync. Better than expectation in low power (USB stable).

Limitation: High noise in shaky conditions – future: adaptive threshold.

13 Challenges and Limitations

13.1 Technical Challenges

- Sensor errors: I2C conflicts (ADXL345/OLED addresses); noisy acceleration data causing false steps.
- Timing issues: Wi-Fi delays in Blynk sync (1-2s lag); debounce timing missed fast walks.

13.2 Resource or Coordination Issues

- Limited components: No battery, so USB-dependent; group coordination delays in coding/testing.
- Time constraints: 3-week timeline squeezed debugging.

13.3 How These Were Resolved or Managed

- Technical: Added I2C scanner; tuned threshold (0.8g) and buffer (15 samples) for noise; optimized timer intervals.
- Resources: Used breadboard for quick fixes; divided tasks (hardware/software) and daily check-ins.
- Overall: Prioritized core features (steps/OLED/Blynk); limitations managed via simulations in Wokwi. Future: Add battery, ML for accuracy.

14 Future Improvements

14.1 Recommendations for Future Versions

- Add daily/weekly step goals with progress bars in Blynk.
- Integrate calorie estimation based on steps and user weight.
- Enable Bluetooth for phone-free data sync.

14.2 Possible Hardware/Software Upgrades

Hardware: Add Li-Po battery and enclosure for true portability; switch to MPU6050 for gyro-based accuracy.

Software: Use machine learning (e.g., TensorFlow Lite) for adaptive step detection; add OTA updates for remote code fixes.

14.3 Enhancing Usability, Efficiency, or Cost

Usability: Voice alerts via buzzer; customizable thresholds in Blynk app.

Efficiency: Low-power sleep mode on ESP32; optimize code to reduce Wi-Fi pings.

Cost: Bulk-buy components or use cheaper alternatives like HC-05 for Bluetooth (~100 BDT savings).

15 Conclusion

The project successfully developed a low-cost step counter using ESP32, ADXL345 accelerometer, and OLED display, achieving accurate real-time step detection at 95% precision during normal walking, with seamless integration to the Blynk app for dashboard updates and remote monitoring. Powered via USB, the prototype costs only 1,200 BDT and demonstrates reliable performance in a compact breadboard setup, fulfilling core objectives of instant local feedback and wireless syncing within a 3-week timeline. Reflecting on the experience, the project was both engaging and demanding, highlighting the ease of rapid prototyping on breadboard while underscoring the value of Blynk for quick IoT features, though time pressures in debugging emphasized the need for early testing. Key learnings include the critical role of I2C sharing and threshold tuning for sensor reliability in technical aspects, alongside collaborative insights like effective task division between hardware and software roles, which accelerated progress through daily check-ins and resolved minor delays efficiently. Overall, this embedded IoT solution proves an affordable gateway to promoting daily physical activity through accessible health tracking.

16 References

1. DIY ESP32-Based Pedometer | Real-Time Step Counting with BMI160 over ESP32 BLE - https://youtu.be/p-draBNhfPU?si=Jkv5GsqRMTw0Df_L
2. ESP32 Step Tracker Tutorial React Native & Bluetooth - <https://youtu.be/ssokvToiYU0?si=Ukbbk2dEStA9FCai9>
3. Step Counter Using Accelerometer - <https://youtu.be/tEFsonIlsfo?si=s2eMha45Ta9Ye736>

17 Appendices

```
#define BLYNK_TEMPLATE_ID "TMPL6H4Jiri03"
#define BLYNK_TEMPLATE_NAME "step counter"
#define BLYNK_AUTH_TOKEN "oCkCpi3s6r2dG_bJDeYWBjI9N_qYZ4I2"
```

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
```

```

#include <Adafruit_ADXL345_U.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

// ===== WiFi Credentials =====
const char* ssid = "ARMAN";
const char* password = "12358123";

Adafruit_ADXL345_Unified accel = Adafruit_ADXL345_Unified(12345);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Step detection variables
float threshold = 0.8;          // Lowered for better sensitivity – tune this!
const int bufferLength = 15;
float accelMagnitudeBuffer[bufferLength];
int bufferIndex = 0;
int stepCount = 0;
bool stepDetected = false;
const unsigned long debounceDelay = 300;
unsigned long lastStepTime = 0;

// Notification tracking
bool milestoneNotified[40] = {false};

BlynkTimer timer;

void sendStepCount() {
  Blynk.virtualWrite(V0, stepCount);
  updateOledDisplay();
}

BLYNK_WRITE(V1) {
  int value = param.asInt();
  if (value == 1) {
    stepCount = 0;
    sendStepCount();
    Serial.println("Step count reset from Blynk app!");
    memset(milestoneNotified, 0, sizeof(milestoneNotified));
    Blynk.virtualWrite(V1, 0);
  }
}

void updateOledDisplay() {
  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 10);
  display.println("Steps:");

```

```

display.setTextSize(3);
display.setCursor(20, 30);
display.println(stepCount);
display.display();
}

void detectSteps() {
  sensors_event_t event;
  accel.getEvent(&event);

  // Raw axes (for debugging)
  //Serial.print("X: "); Serial.print(event.acceleration.x);
  //Serial.print(" Y: "); Serial.print(event.acceleration.y);
  //Serial.print(" Z: "); Serial.print(event.acceleration.z);

  // Magnitude
  float magnitude = sqrt(event.acceleration.x * event.acceleration.x +
    event.acceleration.y * event.acceleration.y +
    event.acceleration.z * event.acceleration.z);

  float deviation = abs(magnitude - 9.8); // Remove gravity

  Serial.print(" | Dev: "); Serial.print(deviation);

  // Buffer and average
  accelMagnitudeBuffer[bufferIndex] = deviation;
  bufferIndex = (bufferIndex + 1) % bufferLength;

  float sum = 0;
  for (int i = 0; i < bufferLength; i++) {
    sum += accelMagnitudeBuffer[i];
  }
  float avgDeviation = sum / bufferLength;

  Serial.print(" | AvgDev: "); Serial.println(avgDeviation);

  unsigned long currentTime = millis();

  if (avgDeviation > threshold && !stepDetected &&
    (currentTime - lastStepTime > debounceDelay)) {
    stepDetected = true;
  }

  if (avgDeviation < threshold && stepDetected) {
    stepCount++;
    lastStepTime = currentTime;
    stepDetected = false;

    Serial.print("*** STEP DETECTED! Total: ");
    Serial.println(stepCount);

    updateOledDisplay();

    // Milestone notification

```



```

    if (stepCount % 5 == 0) {
        int milestoneIndex = (stepCount / 5) - 1;
        if (milestoneIndex < 40 && !milestoneNotified[milestoneIndex]) {
            String message = "Congrats! You've reached " + String(stepCount) + " steps! Keep going!";
            Blynk.logEvent("steps_milestone", message);
            milestoneNotified[milestoneIndex] = true;
            Serial.println("Notification sent: " + message);
        }
    }
}

void setup() {
    Serial.begin(115200);
    delay(100);

    Wire.begin();

    if (!accel.begin()) {
        Serial.println("Could not find ADXL345 sensor! Check wiring.");
        while (1) delay(10);
    }
    accel.setRange(ADXL345_RANGE_16_G);
    Serial.println("ADXL345 initialized.");

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("SSD1306 allocation failed");
        while (1) delay(10);
    }
    display.clearDisplay();
    display.display();
}

```